Lecture 2: Lab Assignment

Christoffer Fink

September 3, 2025



A bit about me

- Christoffer Fink (christoffer.fink@ltu.se, A2310)
- PhD student at CPS (Cyber-Physical Systems)
- Master's student here (Civilingenjör Datateknik, Informations- och Kommunikationsteknik)
- Took this course as a student a few years ago
- This is my third year as a TA on this course.

This lecture

- Lab assignment
  - Brief intro to DHTs and Kademlia (much more next lecture)
  - The assignment
  - Common pitfalls (testing and concurrency)
- The Actor concurrency model (in Elixir/Erlang)

The lab assignment

Implement the **Kademlia** DHT (Distributed Hash Table)

What is a Distributed Hash Table?

DHTs and Kademlia

- Regular hash tables (a.k.a. dictionary or map) map keys to values
  - E.g. Python `dict` or `java.util.HashMap` etc.
- Insert key-value pairs.
  - E.g. `map[key] = value` or `map.put(key, value)`
- Give it a key, and it gives you back the associated value.
  - E.g. `value = map[key]` or `value = map.get(key)`
- Distributed hash tables do the same thing, but it's a distributed system.
- Many networked computers (nodes) work together to implement a hash table.
- Why?

Why a *Distributed* Hash Table?

DHTs and Kademlia

- Why?
- The usual reasons for distributed systems:
  - Performance / scale
  - Availability / fault-tolerance
  - Decentralized
- More details next lecture, including practical use cases.

What is Kademlia?

DHTs and Kademlia

- Kademlia is one particular DHT. (We'll see others next lecture.)
- Node IDs and keys are 160-bit values.
- One key feature is the use of `XOR` as a distance metric.
- To compute the (logical) distance between two IDs/keys, we take their `XOR` and just treat the result as an integer.
- Long common prefix $\implies$ many leading zeros $\implies$ small distance.
- First bit different $\implies$ large distance.

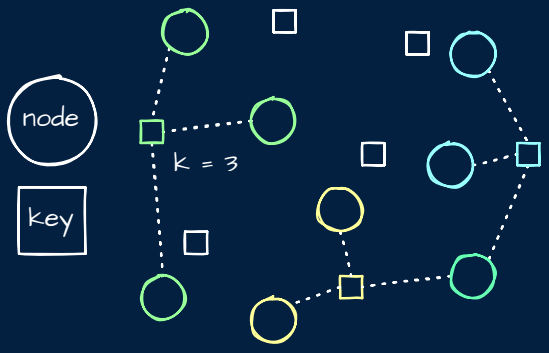
A	B	$A \oplus B$	$A \neq B$	$A = B$
0	0	0	false	true
0	1	1	true	false
1	0	1	true	false
1	1	0	false	true

$$
\begin{aligned}
&01101000\cdots10111001 && \text{ID 1}\\
\oplus\,&01000110\cdots10110100 && \text{ID 2}\\
=\,&00101110\cdots01001101 && \text{distance}
\end{aligned}
$$

What is Kademlia?

DHTs and Kademlia

- A key-value pair is stored by the *k* nodes whose IDs are “closest” to the key.
- When a node wants to store a key-value pair in the network, it has to find the nodes close to it so it can ask them to store the value.
- Similarly, to find the value for a given key, the node has to find the closest nodes* so it can ask them for the value. (*At least one.)



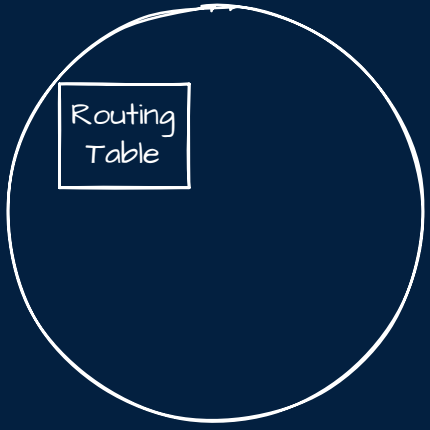
Node lookup in Kademlia

DHTs and Kademlia

- Nodes have a routing table with (limited) information about the network (other nodes).
- When a node wants to find the nodes closest to some ID/key, it picks the best candidates it can find in its routing table.
- Then it asks those nodes, hoping they can suggest even better ones.
- They reply with the closest nodes they can find in **their** routing tables.
- Then the node asks those candidates, and so on.
- Eventually, the node exhausts its list of candidates without finding any that are better.
- Apparently there are no closer nodes than the ones it has found.

Node lookup in Kademlia

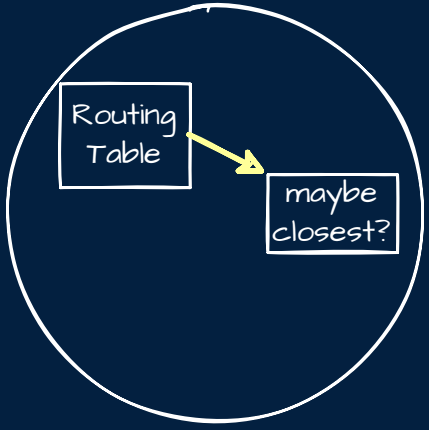
DHTs and Kademlia





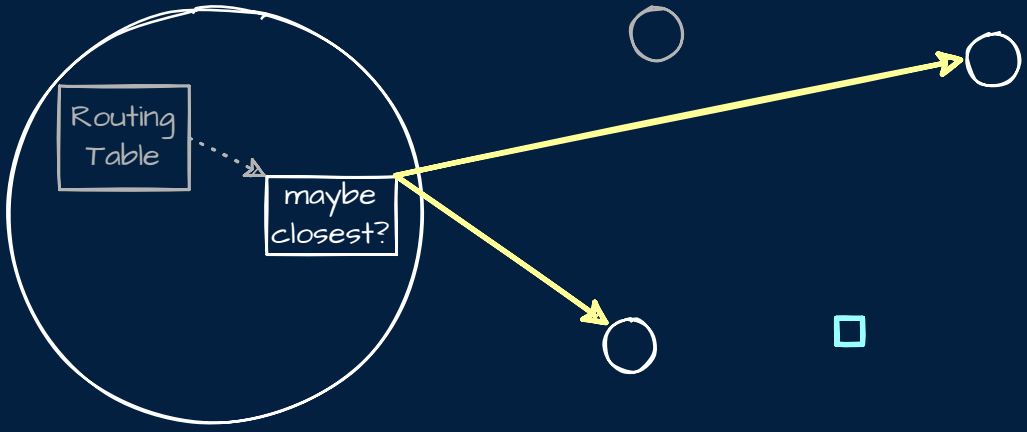
Node lookup in Kademlia

DHTs and Kademlia



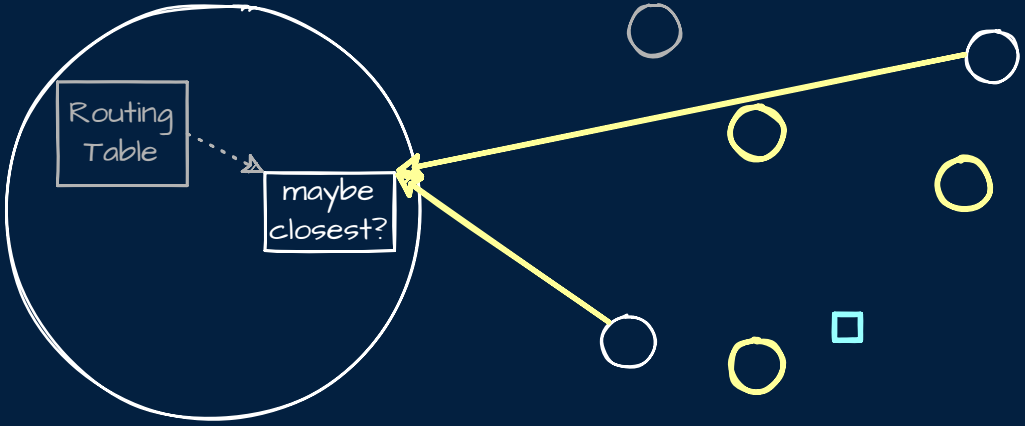
Node lookup in Kademlia

DHTs and Kademlia



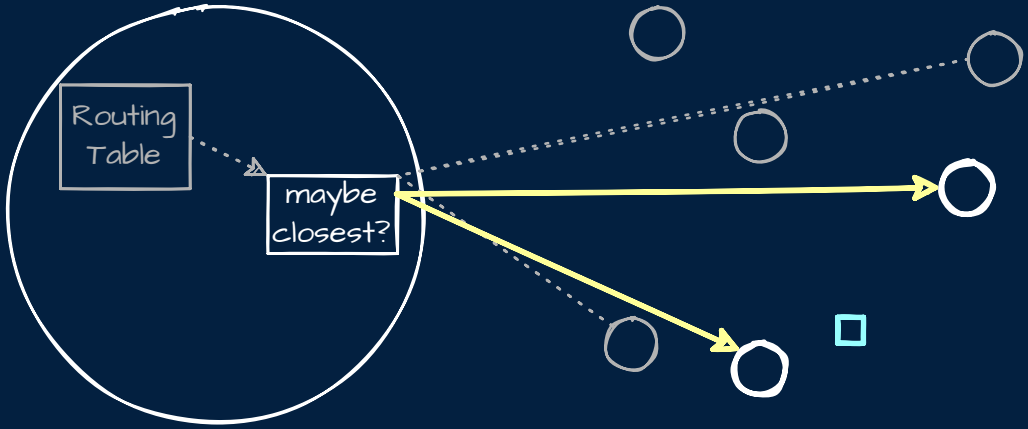
Node lookup in Kademlia

DHTs and Kademlia



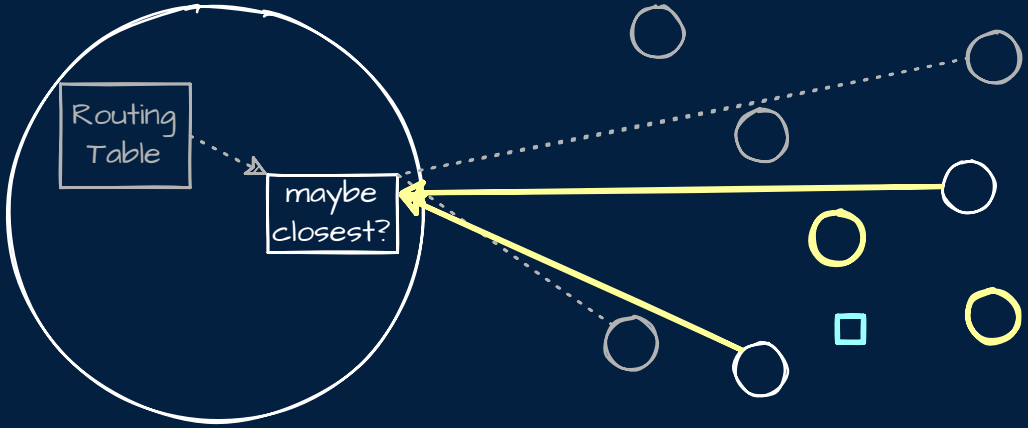
Node lookup in Kademlia

DHTs and Kademlia



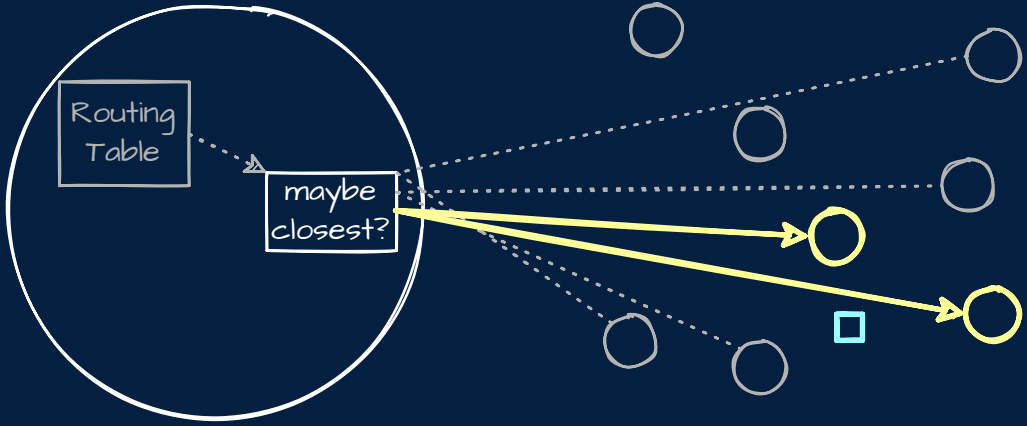
Node lookup in Kademlia

DHTs and Kademlia



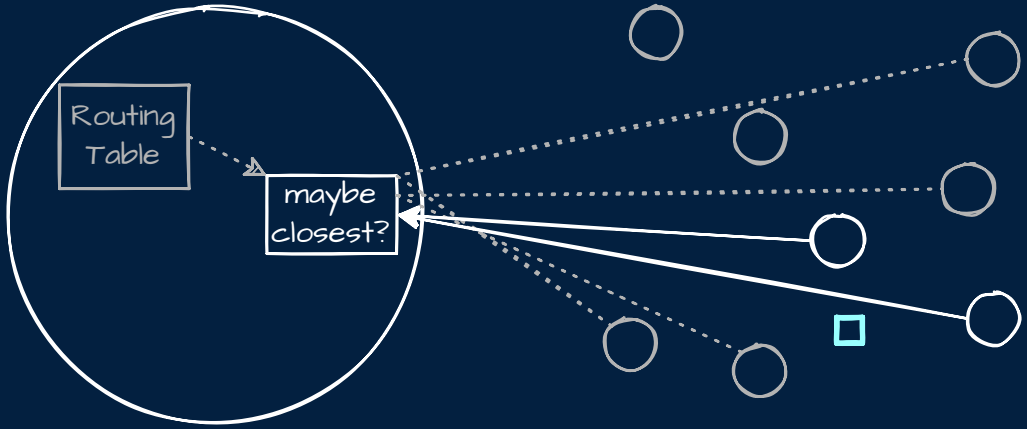
Node lookup in Kademlia

DHTs and Kademlia



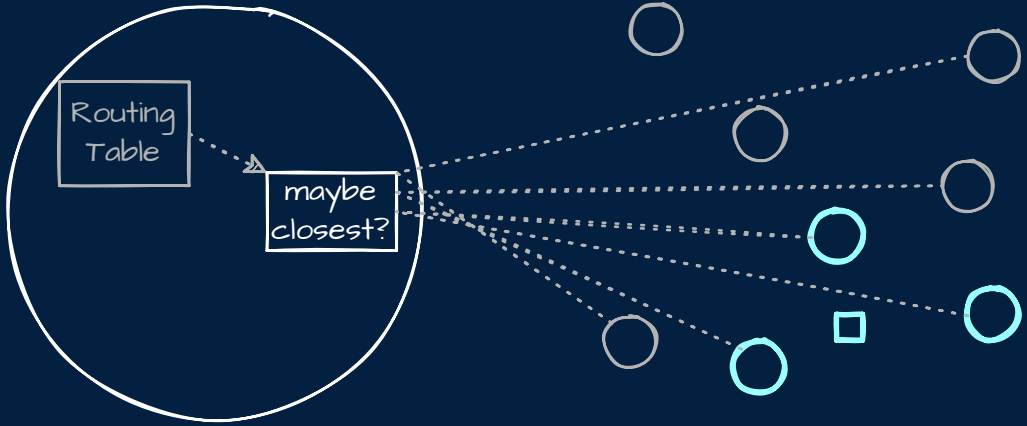
Node lookup in Kademlia

DHTs and Kademlia



Node lookup in Kademlia

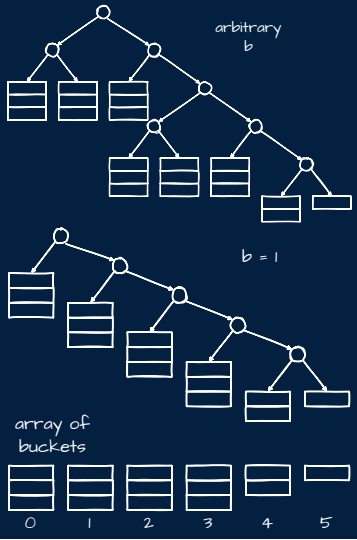
DHTs and Kademlia



Simplified Routing Table

DHTs and Kademlia

- The routing table is a tree structure.
- Leaves are *k*-buckets (lists of up to *k* nodes).
- Buckets are split when they get full, and the tree grows.
- **For the assignment**: a simplified routing table is sufficient.
- **List** of buckets. Much easier:
  - No need to split buckets.
  - No need to traverse the tree; access buckets directly by index.



Next lecture

- This was a very brief introduction to DHTs and Kademlia. Just enough to explain the assignment.
- We'll go into more detail next lecture.
- Until then, you should **read the original paper** that described Kademlia.
- It's pretty short and (mostly) readable.
- Making a serious effort to understand it will make the technical details in the next lecture much easier to absorb.

Programming language and starter code

The lab assignment (how)

- You will be given some Go (a.k.a. golang) code to get you started.
- Go is a nice language for implementing this kind of system, and we encourage you to use it for the assignment.
- You *may* use (almost) any other programming language instead of Go, but we do not guarantee that we can help you if you get stuck. (Check with us if it's OK.)
- One exception: If you use Elixir (or Erlang) I'll be happy to assist.
- *Officially supported languages are **Go** and **Elixir**.*

More practical stuff

The lab assignment (how)

- Work in groups of 3. Groups of 2 *can be OK*. (PhD students work solo.)
- There are 3 “sprints” (0-2) where you show your progress.
- The end result (sprint 2) is what determines how many points you earn.
- It’s a challenging assignment. Make steady progress and don’t leave too much for sprint 2.
- Points for **mandatory** and **qualifying** (optional) objectives.
- You will run your implementation in Docker containers.
- For sprint 0:
  - Focus on getting Docker set up and on understanding Kademlia.
  - Also, start on the report. (Updated for each sprint.)
    - A rough plan (for sprint 1) and sketch of your system.
    - Mundane requirements like group number, member names, git repo.
  - You are not expected to have made any significant progress on your implementation.

Signing up for sprint reviews

The lab assignment (how)

- Sign up in a spreadsheet, which you can find in Canvas.
- Submit your report the day before we meet.
- You will meet either me or Carl.
- We **strongly prefer** that we meet **in person**.
- If some member of your group cannot participate in person, Zoom is an option. Then the whole group is online.

Lab sessions / office hours

The lab assignment (how)

- Many scheduled lab sessions.
- Only 3 of these are sprint reviews.
- For the others, I am available to answer questions.
- I will hang out in our regular Zoom room (same as for lectures).
- You are also welcome to come by my office (A2310).

Common pitfalls

The lab assignment (how)

- Testing
  - Problem: writing tests too late.
- Concurrency (thread safety)
  - Problem: implementations are rarely (if ever) thread safe!

- Johan Kristiansson will tell you much more about testing and concurrency specifically in Go (practical)

Testing

The lab assignment (how)

- Automated tests and a minimum code coverage is a mandatory requirement.
- TIP: Write tests **early**!
- Does that mean TDD (Test-Driven Development)?
- Not necessarily.
- People mean different things by “TDD”, and different variants have different pros and cons.
- That’s how wars get started!

TDD?

Testing

- The TDD slogan is typically “test **first**”
- I deliberately suggested to “test **early**”
- What I **don’t** want you to do is leaving the tests until the end.
- *Don’t **first** finish the implementation and **then** add tests!*

Why write tests?

Testing

- It's nice to have some confidence that certain parts work correctly so you can focus your attention on the work-in-progress.
- Tests help with debugging.
  - Implicitly: you know what parts are thorougly tested and likely work correctly.
  - Explicitly: **Almost** like scripting what you would do with a debugger.
  - Bonus: you now have a test case that you can run repeatedly and whenever you want.
- Automation provides a safety net.
  - Manual testing means manual re-testing after changes.
- **Often**, testable code is also more modular and arguably better.
- Tests are documentation.
  - Ever thought that a code example was more illuminating than a description?
  - Tests are **detailed** documentation. (Should also cover interesting corner cases.)
  - Tests are **executable** documentation. Unlike comments, they never get out of sync with the implementation.
- Bottom line: Tests reduce stress and frustration!

Why write tests *early*?

Testing

- The earlier you write tests, the earlier you can enjoy the benefits.
- **If** you do write tests **first**, it forces you to think carefully about what it is you're actually trying to do.
  - It discourages you from just hacking away blindly until you realize you've been solving the wrong problem.
  - The downside is that it can be difficult to figure out all the details in advance. You may want to do some exploratory coding first. **This is fine!**
- Adding tests when you're done can be very difficult. You might find that you have a big blob that "does something".
- Bottom line: If you add tests at the end, not only does it take effort to cram them into your project, but you won't get any benefit out of them either.

Why test early?

Testing

This happens *every year* (so far):

- I suggest to students to test early. (Like I'm doing now.)
- Some groups get stuck on something that's not quite working right.
- I ask if they have tests for that functionality.
- When they say that they don't, I suggest that it might be useful.
- At the end of the course/assignment they find the tests helpful and regret not writing them earlier.
- Let's break the cycle this year! ;)

Testing advice summary

To summarize, here's my advice:

- Sometimes, writing the test first is **easy**. Then do it.
- Sometimes, writing the test first is **difficult**. Then don't do it.
- Just don't wait too long.

Most importantly: Look at the tests as something that **helps you create the implementation**, not something you add at the end to check that your finished implementation is actually finished (actually works), or because it says in the assignment that you are supposed to have tests.

Concurrency

- Go was designed to have great support for concurrency.
  - The `go` keyword let's you start *goroutines* (lightweight, a.k.a. "green", threads).
  - Threads can communicate via *channels*.
- Johan will cover concurrency in Go in more detail.
- My goal is to illustrate the Actor model using Elixir.
- You will be able to apply the concepts in Go! (And get the most out of channels.)
- Why Elixir/Erlang?
  - While you can do more or less the same thing in Go, Elixir (and Erlang) has support for the actor model deeply integrated into the language. So it's a cleaner illustration.
  - Also, you might get excited about learning one of those languages and maybe even using it in the assignment. ;)

Concurrency

- Roughly: Concurrency $\approx$ multiple things happening at the same time (concurrently)
- Very roughly: Concurrency $\approx$ multi-threading.
- Concurrency can be extremely tricky.
- Go makes it really easy and convenient to run something in a new thread:
  - `go foo() // run foo() in a new thread`
- But the challenge is to make sure these threads to not tread on each other's toes.
- So we can easily achieve multi-threading, but what about thread safety?

Hello World of concurrency

Low-level concurrency

The classic illustration of the most basic thread safety issue is a counter:

```go
var counter int = 0

func inc() {
    counter += 1
}

func main() {
    for range(1_000_000) {
        go inc()
    }
    time.Sleep(5 * time.Second)
    // counter: 835,466
    fmt.Printf("counter: %d\n", counter)
    missed := 1_000_000 - counter
    // missed 164,534 increments!
    fmt.Printf("missed %d increments!\n", missed)
}
```

It looks like ‘ `counter += 1` ‘ is a single (atomic) step. But it’s really two steps, equivalent to:

```go
func inc() {
    // read the current value
    cur := counter
    // write the new value
    counter = cur + 1
}
```

Hello World of concurrency

Low-level concurrency

This means the two operations (read and write) can be iterleaved. For example:

- Thread 1 might read the value n.
- Then, before it writes $n + 1$, thread 2 also reads n.
- Eventually thread 1 and 2 both write $n + 1$ (in some order).
- Now one increment has been lost!
- What if we have N threads instead of 2?

What we would like:

counter=	Thread 1	Thread 2
n	$n_1 := counter$	
$n+1$	$counter := n_1 + 1$	
$n+1$		$n_2 := counter$
$n+2$		$counter := n_2 + 1$

What can happen instead:

counter=	Thread 1	Thread 2
n	$n_1 := counter$	
n		$n_2 := counter$
$n+1$	$counter := n_1 + 1$	
$n+1$		$counter := n_2 + 1$

What went wrong?

Low-level concurrency

- The reason the interleaving matters is that the two threads share (mutable) memory.
- So we get this problem (a data race) when
  - multiple (at least 2) threads can access (read and/or write) the same memory, and
  - at least one of them writes (i.e. mutates the memory).
- *The root cause is **shared** and **mutable** memory.*
- In that case, timing matters, and threads need to coordinate so they read/write at the right time.
- *If there is no shared memory **or** the memory is immutable, then this problem cannot happen.*

Fixing the counter with locks

Low-level concurrency

- Go has *mutex locks* that we can use to fix this problem.
- Solution: identify the *critical sections* and use *mutual exclusion* to ensure only one thread at a time can execute it.

```
var counter int = 0
var mutex: sync.Mutex
func inc() {
    mutex.Lock()    // aquire lock (blocks if another thread has it)
    // ---------- <critical section> ----------
    cur := counter
    counter = cur + 1
    // ---------- </ critical section> ----------
    mutex.Unlock()  // release lock (unblocks a waiting thread)
}
// counter: 1,000,000
// missed 0 increments!
```

Fixing the counter with locks

Low-level concurrency

- This is a very low-level approach.
- Potential advantage: high performance.
- But that depends on granularity.
  - Suppose you have a long function. Do you lock the whole function or just a small section?
  - Suppose you have a data structure with multiple operations.
  - Do you lock the whole data structure or different operations separately?
  - What if they interact? Which combinations are safe? Etc.
  - Might need multiple locks. Different op, different subset. Potential deadlock!
- Just locking everything is (relatively) easy.
- But then you miss out on potential parallelism and lose performance. (Deadlocks?)
- Getting it right is “impossibly hard” – Rich Hickey
- **Alternatives?**

A brief intro to Erlang, Elixir, and the BEAM

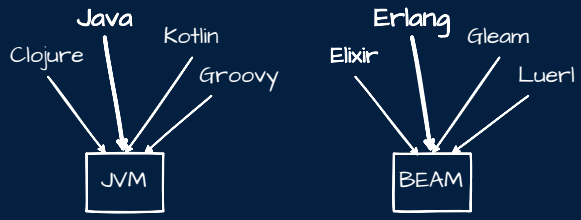
Actor model in Elixir/Erlang

- **Erlang** is a programming language that was developed at Ericsson in the 80s.
- Runs on a virtual machine called the **BEAM** (Bogdan/Björn's Erlang Abstract Machine).
- Specifically designed for building distributed and fault-tolerant systems.
  - Fantastic concurrency support as a by-product.
- **Elixir** is another (newer) language for the BEAM.

A brief intro to Erlang, Elixir, and the BEAM

Actor model in Elixir/Erlang

- You know how there are other languages for the JVM (Java Virtual Machine) than Java?
- Like Groovy, Kotlin, Clojure, and so on?
- Analogy:
  - Erlang $\approx$ Java
  - BEAM $\approx$ JVM
  - Elixir $\approx$ Groovy/Kotlin/Clojure/etc.



Elixir vs Erlang not very relevant.

$|\text{Java} - \text{Clojure}| \gg |\text{Erlang} - \text{Elixir}|$

Abstract actor model

Actor model in Elixir/Erlang

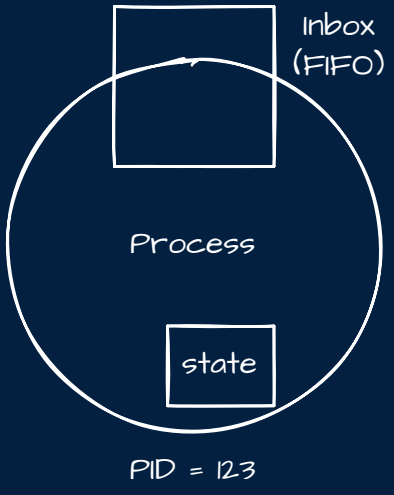
- An **actor** has **private state** and a **mailbox**.
- Actors communicate by sending **immutable** messages **asynchronously** to each other.
- No shared memory.
- Sending messages is asynchronous (non-blocking).

Actor model in Elixir/Erlang

- The actor model is not unique to Erlang/Elixir.
- In fact, the creators of Erlang were unaware of it and rediscovered it independently.
- It is also implemented in various libraries (e.g. the Java library Akka).
- And you can use the same ideas in Go!

Actor model in Elixir/Erlang

- Process $\approx$ actor.
- Each process has
  - its own, private memory (no shared memory)
  - a “mailbox” where messages from other processes are collected (FIFO queue)
  - a unique process identifier (PID).
- Lightweight (similar to Go’s goroutines in terms of resource requirements)
  - Managed by the BEAM (not the OS).
  - Fast to start.
  - Uses little memory.
  - Can have millions running.



A brief intro to Elixir

Actor model in Elixir/Erlang

The basic building blocks are

- `spawn`
  - takes a function and starts a new process that runs it
  - returns the PID (Process ID) of the process
- `send`
  - takes a PID and a message
  - sends the message to the process (also returns it)
- `receive`
  - receives a message that was sent with `send`
  - chooses what to do depending on which pattern the message matches
  - evaluates to ("returns") the last expression

```
pid = spawn(fn -> foo() end)

send(pid, message)

receive do
    pattern1 -> action1
    pattern2 -> action2
    # ...
end
```

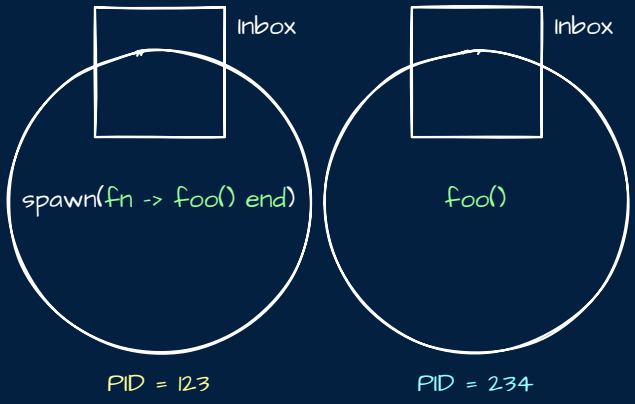
spawn

A brief intro to Elixir

`spawn` creates a new process and returns its PID.

`spawn(fn -> foo() end)` is like `go foo()` in Go.

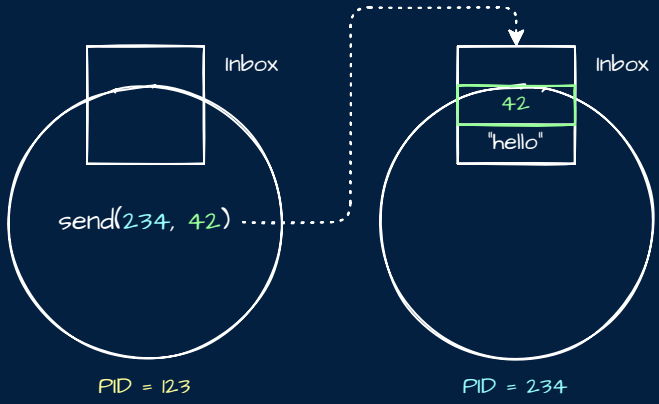
Could also do `spawn(&foo/0)`



send

A brief intro to Elixir

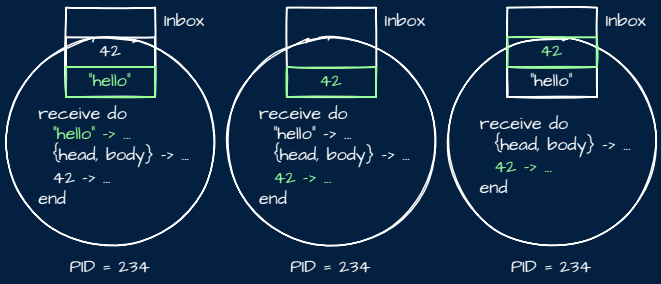
`send(pid, msg)` adds (enqueues) the `msg` in the inbox of the process with ID `pid`.



receive

A brief intro to Elixir

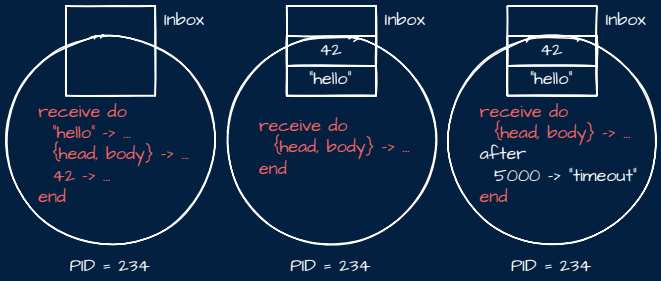
- `receive` tries to consume a message that matches one of the patterns (clauses).
- Tries messages FIFO. Consumes the first message that matches.
- Tries patterns top-to-bottom. Executes the first clause that matches.



receive

A brief intro to Elixir

- `receive` **blocks** if there is no message to receive
- or if no message matches any of the patterns.
- But we can specify a timeout with `after`. A bit like `else`.



The counter example in Elixir

Actor model in Elixir/Erlang

```elixir
def counter_loop(cur) do
    receive do
        :inc ->
            counter_loop(cur + 1)
        {:get, sender_pid} ->
            send(sender_pid, cur)
            counter_loop(cur)
    end
end
```

- Expects two kinds of messages.
- Calls itself recursively (infinite loop).
- State changes by changing the arg in the recursive call.

```elixir
counter_pid = spawn(fn -> counter_loop(0) end)

# spawn incrementing processes
Enum.each(1..1_000_000, fn _ ->
    spawn(fn ->
        send(counter_pid, :inc)
    end)
end)

# give them a second to finish
:timer.sleep(1000)

# ask for the counter value
send(counter_pid, {:get, self()})

# receive the response
receive do
    counter ->
        IO.puts("counted up to #{counter}")
end
```

The counter example in Elixir

Actor model in Elixir/Erlang

- We can clean this up a bit.
- You don't usually see many "naked" `send/receive` or even `spawn`.
- They are hidden inside interface functions in modules.

The counter example in Elixir

Actor model in Elixir/Erlang

```elixir
defmodule Counter do
    def start() do
        spawn(fn -> loop(0) end)
    end

    def inc(counter) do
        send(counter, :inc)
    end

    def get(counter) do
        send(counter, {:get, self()})
        receive do
            value -> value  # XXX: Not ideal!
        end
    end

    defp loop(state) do
        receive do
            :inc ->
                loop(state + 1)
            {:get, sender_pid} ->
                send(sender_pid, state)
                loop(state)
        end
    end
end
```

```elixir
# start counter
counter = Counter.start()

# spawn incrementing processes
Enum.each(1..1_000_000, fn _ ->
    spawn(fn ->
        Counter.inc(counter)
    end)
end)

# give them a second to finish
:timer.sleep(1000)

# get the counter value
value = Counter.get(counter)
IO.puts("counted up to #{value}")
```

The counter example in Elixir

Actor model in Elixir/Erlang

Instead of waiting for some arbitrary amount of time for all increments to finish, we can send a "done" message and proceede after receiving "done" from every process.

```elixir
counter = Counter.start()

parent = self()
Enum.each(1..1_000_000, fn _ ->
    spawn(fn ->
        Counter.inc(counter)
        send(parent, "done")
    end)
end)

# wait for all increments to finish
Enum.each(1..1_000_000, fn _ ->
    receive do
        "done" -> nil # ignore
    end
end)

value = Counter.get(counter)
IO.puts("counted up to #{value}")
```

More complicated data structures

Actor model in Elixir/Erlang

```elixir
defmodule RoutingTable do

    def add_node(rt, node_info) do
        # ...
    end

    def get_closest(rt, key) do
        # ...
    end

    defp loop(state) do
        receive do
            {:add, node} ->
                loop(add(state, node))
            {:closest, key, caller} ->
                loop(closest(state, key, caller))
        end
    end

    # ...

end
```

```elixir
rt = RoutingTable.start()
```

```elixir
RoutingTable.add_node(rt, node_info)
```

```elixir
nodes = RoutingTable.get_closest(rt, key)
```

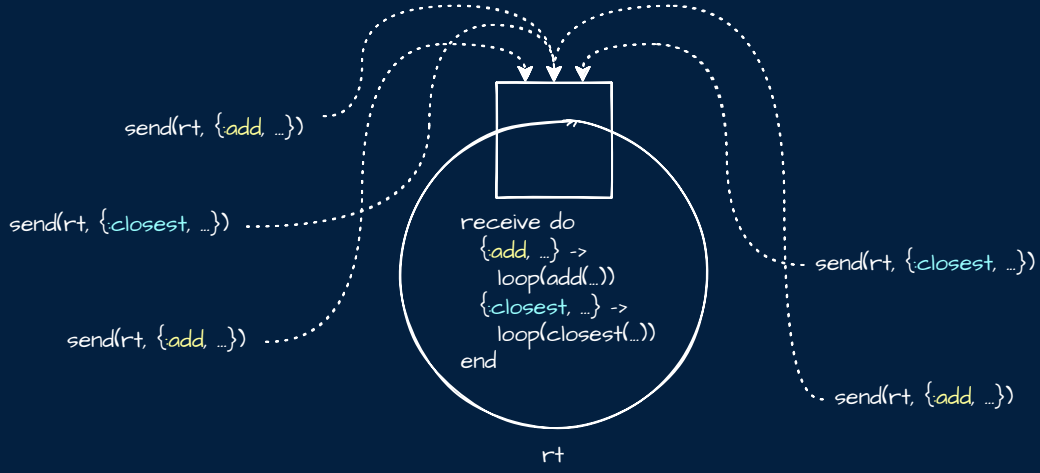
```elixir
RoutingTable.add_node(rt, node_info)
```

```elixir
nodes = RoutingTable.get_closest(rt, key)
```

```elixir
RoutingTable.add_node(rt, node_info)
```

More complicated data structures

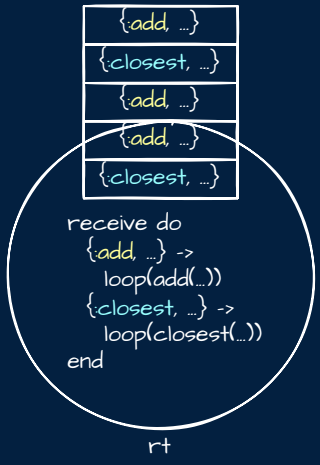
Actor model in Elixir/Erlang



More complicated data structures

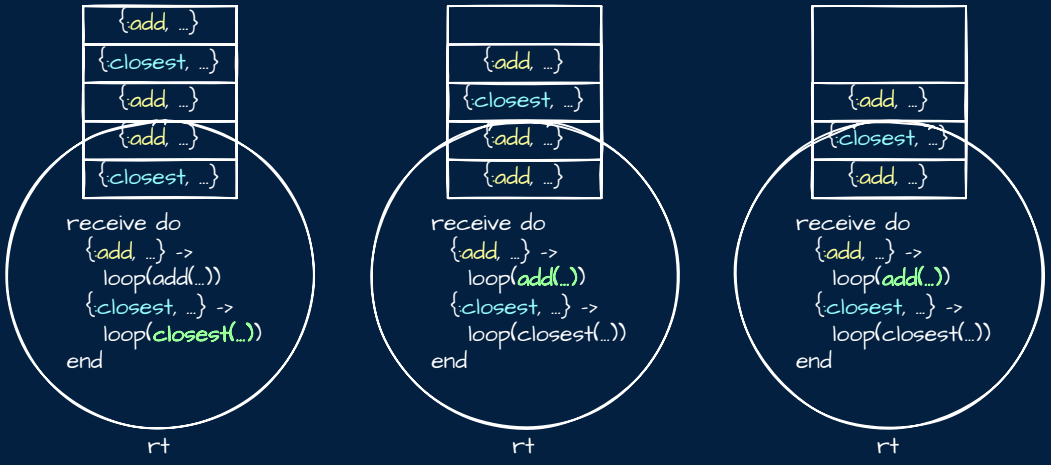
Actor model in Elixir/Erlang

The messages will arrive in the inbox in some order and be handled sequentially.
Processes work in parallel and send requests concurrently. But the inbox serializes them, and each process handles one request at a time.
(No concurrency inside a process!)



More complicated data structures

Actor model in Elixir/Erlang



General pattern

Actor model in Elixir

- Encapsulate state in a process.
- Handle messages in a loop. (Like an event loop.)
- Some requests are asynchronous (fire-and-forget), some are synchronous (caller waits for a result).
- So common that there are templates like GenServer (generic server).
- Template: you define callbacks.

- Goes beyond hiding `spawn`, `send`, and `receive` in interface functions. (Hides the message loop!)
- Don't need to manually roll your own every time.
- Also adds many nice features.
  - Properly matching requests and responses.
  - Registering processes.
  - Linking (monitoring) processes.
  - Timeouts.
  - Asynchronous replies. (Don't block the message loop when computing the result.)
- Probably good to do it yourself first before relying on automagical functionality.

More features (teaser)

- The focus was concurrency and the actor model.
- Erlang / Elixir / the BEAM have many more cool features (for distrubution and fault tolerance).
- Distribution. (Sending messages between remote processes.)
- Hot update and rollback. (No need to restart the whole application.)
- Supervision trees. (Automatically restarting crashed processes.)

More about Elixir?

- The Soul of Erlang and Elixir
  - https://www.youtube.com/watch?v=JvBT4XBdoUE
- BEAM Concurrency in Action
  - https://www.youtube.com/watch?v=A4x6IfceJCM
- If there is interest, we could use a lesson to talk a bit more about Elixir.
  - More Elixir code examples?
  - Relating the actor concepts back to Go?

Recap

The lab assignment

- Working in **groups of 3** (or 2) (PhD students work solo)
- Implement **Kademlia**
- Deploy in **Docker** containers
- Explicitly allowed languages are **Go** and **Elixir**. (For others: ask!)
- Present your work in **3 sprints**, remember to **sign up**
- Submit your report the day **before** we meet.
- The final sprint is what counts (in terms of your grade/points)
- Tip: **Test early**
- Tip: **Avoid mutex locks** (prefer higher-level tools)

Next lecture

Next lecture, you should already know a bit about Kademlia.

- Roughly how the routing table works (e.g. what k-buckets are and how to find appropriate nodes in the routing table given some target ID/key).
- Roughly how the node lookup algorithm works.
- What (purpose) the lookup algorithm is used for.
- How nodes join the network.
- How do nodes learn about the state of the network?


LULEÅ
UNIVERSITY
OF TECHNOLOGY